SEPM Project Review 1

TEAM MEMBERS-

AMRIT SINHA (RA2311026030097)

ABHINAV SINGH (RA2311026030068)

AKSHAT VISHNOI (RA2311026030100)

PolyScriptOCR — Multilingual OCR & Handwriting Recognition

Semester Project — 3rd Year B.Tech | First Review

Transforming handwritten and printed multilingual documents into searchable, editable digital text through advanced machine learning.

PROJECT ROADMAP

Agenda & Project Overview

01

Business Motivation

Understanding the problem and target users

02

Stakeholders & Scope

Key players and development approach

03

System Architecture

Models, pipeline, and technical design

04

Feasibility Analysis

Tech stack and resource validation

05

Timeline & Risk Management

Deliverables, milestones, and mitigation strategies

This review presents our progress on building a robust, multilingual OCR system capable of recognizing both handwritten and printed text across multiple scripts.

Business Case & Problem Statement

Target Users

Students digitizing handwritten lecture notes, researchers archiving multilingual manuscripts, and educational institutions building searchable document repositories.

Core Pain Points

- **Handwritten notes remain unsearchable and difficult to organize**
- **Existing OCR tools struggle with multilingual and mixed-script documents**
- **Low accuracy rates waste time on manual corrections**
- **Limited access to affordable, flexible OCR solutions**

Value Proposition

PolyScriptOCR delivers searchable digital archives, saves hours of manual transcription, and enables automated document workflows across languages.

Stakeholders & Process Model

Primary Stakeholders

End users: students, educators, researchers who directly interact with the OCR system

Secondary Stakeholders

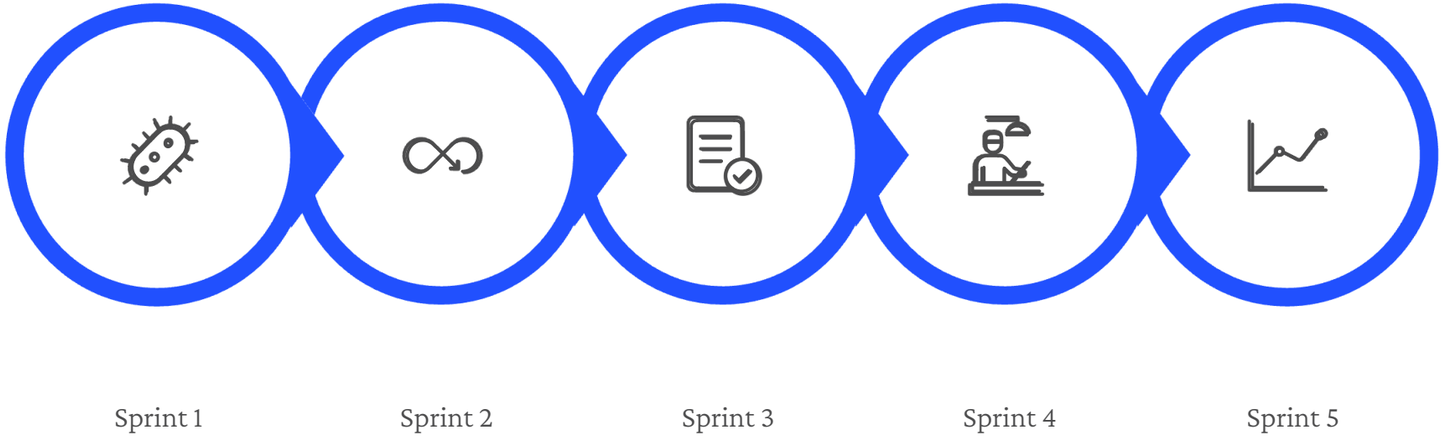
Development team, project evaluators, and technical reviewers ensuring quality

Tertiary Stakeholders

Academic institutions, funding bodies, and future research communities

Development Approach: Agile–Scrum

We adopted an iterative Agile methodology with 8 two-week sprints, enabling continuous experimentation and rapid adaptation—essential for ML model refinement.



Each sprint focuses on a specific milestone: data preparation, model training, backend integration, UI development, and testing cycles.

REQUIREMENTS

SRS — Scope & Requirements

Functional Requirements

1. Document Upload: Support for image formats (PNG, JPG) and PDF files
2. Script Detection: Automatic identification of writing systems (Latin, Devanagari, Arabic, etc.)
3. OCR Recognition: Accurate extraction of both handwritten and printed text
4. Editable Output: Users can review and correct recognized text in-app
5. Export Options: Download results as TXT, DOCX, or searchable PDF

Non-Functional Requirements

- Accuracy Targets: Character Error Rate (CER) < 5%, Word Error Rate (WER) < 10%
- Performance: Inference latency under 3 seconds per page on standard hardware
- Security: Encrypted file uploads with automatic deletion after processing
- Portability: Containerized deployment for cross-platform compatibility
- Scalability: Modular architecture supporting future language additions

These requirements form the foundation of our acceptance criteria and testing strategy.

Feasibility & Tech Stack

Technical Feasibility

Leveraging PyTorch and pretrained OCR models with GPU acceleration ensures rapid prototyping and training. Open-source frameworks reduce development risk.

Economic Feasibility

Utilizing publicly available datasets and controlled cloud GPU usage keeps costs within academic project budgets while maintaining training quality.

Operational Feasibility

Modular design with Docker containerization ensures easy deployment, maintenance, and future enhancements across different environments.

Technology Stack

ML Models

CRNN & Transformer-based OCR architectures

Backend

FastAPI for high-performance REST endpoints

Frontend

React for responsive user interface

DevOps

Docker, Label Studio for annotation

Data, Annotation & Training Strategy

Dataset Foundation

We curated a diverse corpus combining public handwritten datasets (IAM, RIMES) with printed multilingual text collections, ensuring broad script coverage and style variation.

Annotation Pipeline

Line-level JSON annotations with bounding boxes and ground-truth transcriptions. Label Studio accelerates manual verification of challenging samples.

Training Methodology

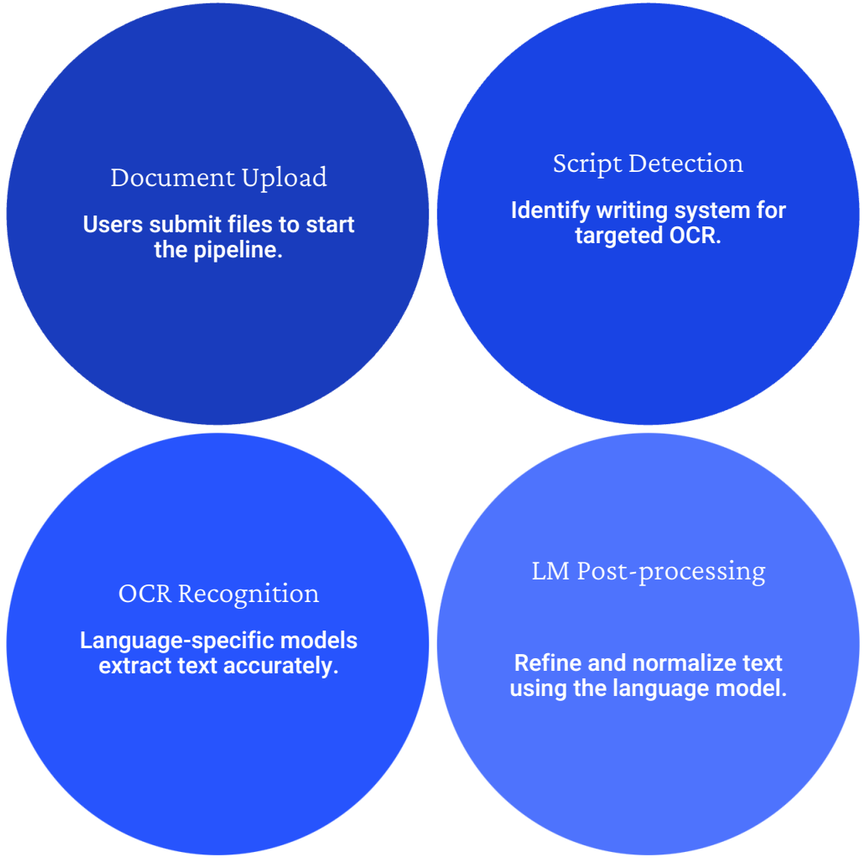
- Augmentation: Noise injection, geometric skew, blur, and contrast variations simulate real-world document conditions
- Loss Function: Connectionist Temporal Classification (CTC) for sequence alignment without explicit segmentation
- Optimizer: Adam with learning rate scheduling for stable convergence

Evaluation Metrics

Character Error Rate (CER) and Word Error Rate (WER) measured on held-out test sets across multiple languages and writing styles.

ARCHITECTURE

System Architecture & Model Pipeline



Our modular pipeline routes documents through specialized components: script detection first identifies the writing system, enabling language-specific OCR models to process text with optimized accuracy. Post-processing with language models corrects common recognition errors and improves output quality.

Upload Module

Accepts images/PDFs, performs preprocessing (deskewing, binarization)

Script Detection

CNN classifier identifies writing system with 98%+ accuracy

OCR Engine

Ensemble of CRNN models specialized per script family

Post-Processing

Statistical language models refine output and fix common errors

THANK YOU